

Training Computer Use Agents to Assess the Usability of Graphical User Interfaces

ANONYMOUS AUTHOR(S)

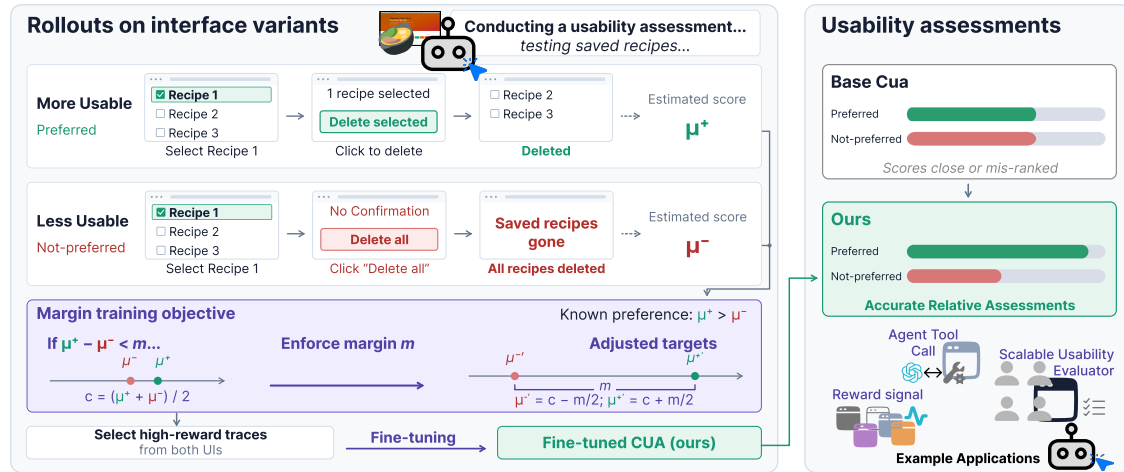


Fig. 1. Our method fine-tunes computer use agents (CUAs) to perform accurate usability assessments by first collecting rollouts on interface variants and enforcing a margin training objective. The resulting CUA also has the potential to enable three HCI applications surrounding scalable usability evaluation and interface design.

Recent HCI research leverages AI agents to assess graphical user interfaces (GUIs) usability, but current agents still struggle. Computer use agents (CUAs) that are used in these assessments are predominantly developed using objective task trajectories with verifiable outcomes and lack an operationalized definition of usability and necessary training data. This work presents a novel training objective that operationalizes usability to fine-tune CUAs to assess GUI usability through relative comparisons between interface variants. We apply our method to CUA models using a large-scale dataset of fully interactive UIs paired with usability labels mixed with human preferences. Two empirical evaluations across real and synthetic GUIs show that our method *i*) improved relative usability assessment accuracy across three off-the-shelf CUA models, *ii*) enabled our best model to outperform proprietary baselines, and *iii*) produced grounded usability assessments. Our work lays a data-driven foundation for agent-based usability assessment, enabling exploration of new forms of scalable usability evaluation and interface design.

ACM Reference Format:

Anonymous Author(s). 2026. Training Computer Use Agents to Assess the Usability of Graphical User Interfaces. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (CHI '27)*. ACM, New York, NY, USA, 29 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

1 Introduction

The concept of *usability* captures how well an interface supports users in accomplishing their goals. Usability assessments, in turn, provide a basis for improving interface design. Human–computer interaction (HCI) has traditionally relied on varying and disparate empirical measures of usability through usability studies [12, 15, 49, 52] and expert evaluations [31, 53]. Without a standardized definition of usability, these traditional assessment strategies lack the ability of at-scale deployment.

Usability is difficult to operationalize as it is inherently subjective and largely dependent on individual experience. For example designers have shown large disagreements on which design issues users surface [14] and UI preferences vary depending on user groups. Even when following expert-written heuristics, individuals will vary in identified issues [53], as what may be a problem for one user may not even be a concern for another. Prior HCI researchers have used computational models to predict how people will interact with interfaces, such as movement and task execution [16] or the perceptual, cognitive, and motor processes [3, 9] underlying interactions as a way to encode human notions of usability. However, these models typically relied on explicit representations of the task, user, or environment, making them difficult to apply broadly across real-world interfaces.

AI agents powered by large language models (LLMs) offer a promising way to move beyond these limitations. Specifically, computer use agents (CUAs) are able to interact with a computer in a similar way as humans do [27, 28]. By combining broad pretraining with the ability to operate directly on rendered graphical user interfaces (GUIs), CUAs generalize across a much wider range of interfaces without requiring a custom model for each one. Recent HCI work has begun to use LLM-based agents for usability evaluations and related design tasks, though using diverse conceptions of usability. Duan et al. [14] used LLMs to offer design critiques through manually crafted guidelines, while others [27, 32, 47, 99] have used them to conduct usability tests. However, unlike earlier models grounded in empirical observations of human interaction, CUAs are trained on large-scale data that are curated or synthesized without explicitly modeling real human behavior or usability. This can lead to hallucinations or incorrect predictions of human behavior [25, 46, 99].

To align CUAs for GUI usability assessments, we propose to operationalize usability through *relative comparisons* using synthetic interface variants. One way to scale usability metrics and provide interaction data needed to fine-tune CUAs is by manipulating an interface to inject defects, as the variant is now *less usable*.

We contribute to data-driven assessments of UI usability by developing a novel training objective for CUAs to more accurately assess GUIs. In that vein, we fine-tune CUA models using relative comparisons of interface variants. To support this training, we create uxWeb, the first large-scale dataset of 2,586 fully interactive GUIs paired with usability labels (plain vs defect-augmented) mixed with human usability judgments. Labels are obtained through a synthetic augmentation pipeline that instructed coding agents to inject common usability errors into websites by modifying their source code. We use our method and dataset to train three off-the-shelf CUA models to improve their usability assessment capabilities (e.g., overall models should assess UIs with known defects as *less usable*).

We conduct two separate evaluations to validate our approach. First, we evaluate whether our method improves usability assessment across three off-the-shelf CUA models, finding consistent improvements (22.6% in AUC) over their corresponding base models. We also compare our best-performing model against strong proprietary and open-source baselines, finding that it outperforms much larger models by 18.1%. Second, we show that improvements in usability assessments resulting from our method were more grounded in observations from their navigation traces on real and

synthetic GUIs. We discuss how our method can be used in downstream applications such as scalable simulated usability testing, agent-assisted user interface design, and reward signal for generative UI techniques.

To summarize, we make the following contributions:

- (1) We create and will publish uxWeb, a large-scale dataset of fully interactive clones of original websites and their defect-augmented counterparts, annotated with usability labels and judgments by designers.
- (2) We introduce a novel method for fine-tuning computer use agent (CUA) models to make accurate usability assessments and show that it provides consistent improvements over corresponding base models, enables our best-performing model to outperform larger proprietary models, and is accompanied with more grounded usability assessments on real and synthetic websites.

2 Related Work

Our research builds on existing work surrounding UI usability evaluation and user simulation by encoding human notions of usability into computational models. We contextualize our work by providing an overview of UI usability evaluation, user simulation, and computer-use agents.

2.1 User Interface Evaluation

Traditionally, UI evaluation has relied on experts to conduct usability studies with target users [51] or heuristic evaluations using a set of guidelines [53]. While expert-driven evaluations can offer rich and targeted feedback, these approaches are costly and only capture the opinions of a small number of evaluators, remaining difficult to scale. As a result, crowdsourced approaches or “arenas” have been used to incorporate feedback from larger numbers of non-experts [48] and are widely used to benchmark the design performance of AI models on design tasks [7]. These approaches enable more rapid feedback, enabling designers to improve UIs for the user experience. Other solutions have focused on modeling human behavior [20, 21, 57] to assess and adapt UIs.

Simultaneously, UI evaluation is not limited to generalized usability; other types of UI evaluations exist aimed towards improving specific aspects of usability. One particular focus is to determine and taxonomize dark patterns [11, 26], or UIs intentionally design to mislead users. Others threads of research have focused on detecting design defects [82, 90] or accessibility issues [17, 98] and even eliciting design preferences that may impact usability for different user groups. Our work extends existing work in user interface evaluation, aiming towards providing automated usability assessment in a data-driven manner.

2.2 User Simulation

One approach of improving automated UI evaluation is to accurately simulate users of those interfaces. Early work in HCI and cognitive psychology approximated UI usability by combining cognitive and physical measurements (e.g., reaction time and motor speed) with interaction traces to approximate time needed to complete tasks. ACT-R [3], incorporated richer models of human cognition, such as memory, attention, and learning to more realistically simulate these user-interface interactions. Together with interaction and task models [60], these user models have been employed to automatically design and evaluate UIs, through approaches collectively known as model-based UI development [62]. However, these approaches largely relied on hand-crafted models and manually specified rules rather than data-driven learning of user behavior.

Since LLMs and other foundation models are trained on large volumes of human-written text and artifacts, researchers hypothesized these models to be effective simulations of human responses and interactions [29, 59, 80], also applying

these approaches to simulate UI feedback. Recent work has explored prompting LLMs with UI screenshots, personas, and design guidelines to simulate comments from end-users [86] and experts [14, 99]. Since usability evaluation often depends on interface interaction rather than static screenshots, researchers have also developed LLM-based agents and crawlers capable of navigating and operating applications as another method of simulating users. Computer use agents (CUAs) have been used to test the functionality of GUIs and identify bugs through “crawling” [22, 91]. Another avenue has been to incorporate these systems into evaluation protocols, such as automated judges [41] or supporting expert-driven evaluation [32, 47, 98].

LLM-based user simulation relies on the assumption that LLMs are accurate models of users, but several works have shown mismatches in human ratings [67], responses [36], and assessments [82]. This suggests a need to incorporate additional training objectives or additional data to refine off-the-shelf models. Oulasvirta et al. [58] theorized that human users behave as bounded rational agents, and their behavior can be approximated by reinforcement learning training objectives. Subsequent work applied these techniques to learn user simulations for typing [70], VR selection [34], and menu navigation [50]. Others have fine-tuned LLMs using human data such as feedback [14, 82, 96] and interaction traces [10] so that their behavior more closely aligns with real users. Our work combines both approaches to fine-tune CUAs for usability testing, with the goal of better aligning their assessment scores and navigation behavior with human users.

2.3 Training Computer Use Agents

Foundation models are promising for UI understanding because of their pretraining on large amounts of web data and text gives them strong capabilities for interpreting interfaces and user instructions. Many approaches for building CUAs prompt foundation models with representations of a target UI and along with the model’s context window and a goal. CUAs can rely on text-only LLMs by converting UIs into intermediate representations (e.g., pseudo DOM) to execute actions [10, 78] or they can use multimodal models [54], where they integrate visual prompting techniques to “ground” their actions. While prompting methods can be effective, they rely heavily on the pre-existing capabilities of the underlying foundation models.

To address this issue, later work has explored learning-based approaches that use demonstration trajectories to train or fine-tune CUAs. For example, Mind2Web combined manually annotated web trajectories with fine-tuned action prediction models [10], while WebGUM more directly framed web navigation as offline multimodal training over a large corpus of demonstrations [18]. Other systems such as UI-TARS and OpenCUA trained end-to-end GUI agents on large action-trace datasets and human computer-use demonstrations [64, 79], and provided automated judges to assess the quality of web development [41]. Another learning-based approach to improving CUA performance is by designing new training objectives or applying reinforcement learning in online environments. Examples include AndroidEnv, which provides an interactive mobile environment for agent training [76], and more recent methods such as DigiRL, WebRL, and EvoCUA, which improve agents through online interaction, reinforcement learning, or large-scale synthetic rollouts [8, 63, 89]. As a result, most CUAs are trained for task completion, e.g., following a user’s natural language command.

3 Dataset of Interface Usability

To explore a data-driven approach for usability assessment capabilities of machine learning models, we first describe our approach to building uxWeb, a large-scale dataset of 2,586 fully interactive websites paired with usability labels and human judgments. Since a large-scale dataset with the location and severity of real-world usability flaws is not readily

available, uxWeb is constructed through a combination of synthetic replication of popular websites, principled data augmentation, and human expert ratings.

3.1 Synthetic Websites

uxWeb consists of sets of synthetically generated website clones with usability labels created from synthetic augmentation and human labeling. We describe the *i)* website generation process to create *plain clones* of existing sites, *ii)* synthetic usability augmentations to create a set of *defect-augmented* counterparts for each plain clone, and *iii)* our protocol for collecting human ratings.

3.1.1 Website Generation. We first generated clones of popular websites drawn from Mind2Web, a standardized web agent navigation benchmark [10] that contains 137 websites, across 5 high-level domains (e.g., shopping, entertainment) and 30 subdomains. Generating standalone “clones” of websites made them easier to deploy, allowed synthetic control of website behavior (e.g., through code edits) and reduced barriers to agent use (e.g., login credentials).

To create these clones, we used the Claude Code agent [5] configured with GLM 4.7 [92] as the backend endpoint for cost efficiency. At the time of our experiments, Claude Code was the best performing coding agent, and GLM 4.7 was the best-performing open source code generation model [7]. We provided GLM 4.7 a bare bones React.js template, to avoid requiring the model to generate “boilerplate” configuration code. The coding agent was then prompted to replicate an existing site from Mind2Web based on its name and a description of its functionality. It was also prompted to ensure that the website clone was fully functional, self-contained, and with mocked backend flows that represented the real website as closely as possible. Placeholder assets such as images were retrieved using an online API that was queried with model-generated alt-text and mock data that was self-generated by the model. The full prompt is provided in Appendix A. Because many of the websites are popular, the coding agent was often able to reconstruct their general interaction flows and common tasks from prior knowledge alone, without requiring screenshots of the reference site. Our goal was not to produce clones that matched the original websites exactly in terms of functionality or appearance, but instead to generate a diverse distribution of realistic websites for our experiments. Once website generation finished, the model also created a txt file documenting all the user flows it had implemented. We used this file for quality control, checking if the first few clones generated had realistic and representative user flows implemented before continuing to generate the rest of our dataset. We also used this file when injecting usability defects. Each website clone took 5-7 minutes for GLM 4.7 to fully implement. The 879 websites required approximately 102.5 machine-hours to generate, which was distributed over the course of a month.

Since synthetic data generation can introduce hallucination, such as the model claiming to have implemented a user flow it did not, or introduce defects due to probabilistic outputs, we cloned each of the 137 Mind2Web sites eight times. We performed two additional rounds of quality checks resulting in a total of 879 plain website clones. The first involved creating a script that would check if the website could be rendered in Playwright, a headless Chrome browser. If the resulting screenshots were blank, the website was removed from our dataset. The second quality check occurred during our training procedure, which we discuss in §4.3.2.

3.1.2 Usability Defect Injection. Like their real-world counterparts, our synthetically-cloned websites were not free of usability defects but the type, location, and severity of these defects remains unknown for both real-world sites and our synthetic ones. To support modeling and benchmarking, we developed an approach to inject *known* usability defects into each of our plain clones in a programmatic manner to create sets of *defect-augmented* versions for each plain synthetic clone. This augmentation process allowed us to measure model performance on a known (but possibly

incomplete) list of usability flaws. This process also resulted in a *relative* ranking of website usability between *plain* and *defect-augmented* sites, which we later used to calibrate absolute score prediction through our training objective.

To design these usability defects, we reviewed various usability and design guidelines [6, 23, 42, 51]. Ultimately, we chose to adapt Ben Shneiderman’s 8 Golden Rules of Interface Design [71] for their relevancy to our task and ability to operationalize¹. Below, are the eight types of usability defects inspired by these principles:

- **Consistency:** Consistent sequence of actions should be required in similar situations (e.g., identical terminology used in prompts, menus, help screens, consistent layout, capitalization, fonts).
- **Feedback:** For every user interaction there should be interface feedback. Feedback should be proportional to frequency and importance of actions.
- **Dialog:** Dialogs should yield closure and should have a clear beginning, middle, and end.
- **Prevention:** The interface should be designed so that a user cannot make serious errors; erroneous actions should leave the interface state unchanged.
- **Control:** Users should feel as if they are in control of the interface; they should not get unexpected surprises or have tedious data-entry sequences.
- **Reversal:** As much as possible, actions should be reversible.
- **Memory:** Avoid designs that require users to remember information when moving from one display to another.
- **Hierarchy:** Important elements central to the task should be emphasized and easy to find.

We automated our usability defect injection in a similar manner to our initial website generation. For each existing plain clone, we prompted our coding agent to inject code changes to introduce defects that violated one of our usability principles above. We provided GLM 4.7 the codebase for the existing website and the usability principle to violate, then instructed it to introduce changes that would violate the user flows documented in the generation step. The coding agent was instructed to keep all changes contained in the `src` folder and not introduce changes that would break website functionality. We injected usability defects into 388 plain sites. Usability defect injection took 7-10 minutes per principle, taking approximately 284.5 hours to generate our 1,707 defect-augmented sites. The prompt for usability defect injection is in Appendix A.

To validate our usability defect injection resulted in *i*) sites that were less usable than the plain counterpart and *ii*) contained defects corresponding to the injection type (i.e. usability principle), we adopted the procedure from Zhang et al. [94] on a subset of uxWeb. Three authors blindly inspected 5 plain sites and a random sample of their defect-augmented counterparts for a total of 47 sites. We found that for 75% of sites, there was unanimous agreement that defect-augmented sites were less usable. For 73% of the sites, there was at least two-thirds agreement that the type of flaws the defect-augmented site contained was the same as the category that was injected.

uxWeb was randomly partitioned into training, validation, and testing splits based on unique website name. We reserved one website per each unique subdomain (30 total) for our test set. 15% of the remaining websites were used for validation, resulting in 92 unique websites and 15 unique websites for the training and validation splits respectively. In total we had 1,573 clones for training, 199 clones for validation, and 393 clones for testing.

3.2 Designer-Rated Usability

Our synthetic approach supports the scalable generation of potentially large numbers of training examples, which is beneficial for model training. However, the usability defects injected to create our defect-augmented sites may not

¹These principles also have significant overlap with Nielsen’s Heuristics [51].

capture usability flaws that were present in the original website clones or reflect human perception of usability, limiting the generalization of models trained on this data. Therefore, we collected an additional split of data labeled by human designers.

3.2.1 Designer Rating Procedure. To collect designer ratings of these clones, we constructed a *model arena*. Model arenas are interfaces commonly used for LLM feedback collection and benchmarking [7, 44], and they require participants to rate multiple candidate outputs for the same input prompt. In our case, we developed a feedback interface where two plain websites *without* intentionally-introduced defects corresponding to the same source website and prompt were compiled and displayed side-by-side in `iframe` elements, allowing experts to use the websites while preserving interactivity. The interface also contained a button that required users to expand the `iframe` for more detailed inspection and access the website using the full resolution of their browser window.

We recruited 17 participants at university level or higher with design background (i.e. university courses or work experience) to partake in our study to rate 30 preference pairs. After navigating the functionality of each site, participants were asked to provide a response that indicated which of the two sites were more usable. Participants were instructed that the sites were clones of existing sites and that placeholder data was used. Participants were recruited through word of mouth or university Slack channels. Our study protocol was determined as exempt by our institution’s IRB. Study details can be found in Appendix C.

In total, we collected 510 preference pairs, which spanned 428 unique plain website clones. Using server logs, we found that participants spent an average of 54.83² seconds on each pair and clicked on the `iframe` window an average of 4.65 times and on an average of 3.05 elements before their final response. This suggests that participants spent effort in testing website flows before making assessments. Nevertheless, when we computed inter-rater agreement for a subset of pairs with multiple labels, we found an agreement rate of 0.308 (Krippendorff’s α). We expected moderate-to-high levels of disagreement, as the websites being compared were all generated from the same model and same prompt, and were therefore more likely to contain similarities. This also mirrors results found by other prior research [14, 48, 61, 82, 85], which found high levels of subjectivity in design-related evaluation.

4 Training Approach

In this section we provide an overview of our novel training objective to train computer use agents to *i*) prioritize exploration of important UI flows and *ii*) and accurately predict usability scores that match ground-truth judgments. We also describe how we use this approach to train CUAs and our agent formulation.

4.1 Problem Formulation

Our resulting CUAs are designed to perform GUI usability assessment in the same way humans do, through visual perception and clicking. While research has shown that having access to hidden semantic information like the HTML or DOM allows web agents [19, 28, 77] to more effectively find information and executing tasks, we chose to formulate the CUA to operate more generally on any type of GUI, using the same visual reasoning a human would typically undergoes (e.g., item search). Based on its initial observation of the input UI, the CUA predicts interaction flows that are important to overall usability (e.g., room booking for a hotel site) and prioritizes important user flows to check within a given interaction budget (e.g., 50 actions). After the navigation phase, the CUA reviews its interaction session and reasons

²We removed outliers based on z-score with $z > 3$ in cases where the participant may have left the study tab open before coming back to complete a preference rating.

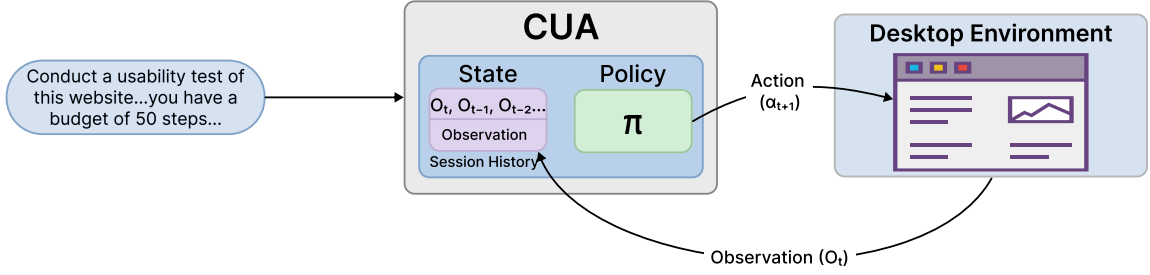


Fig. 2. Our CUA architecture for GUI usability assessment. At each timestep, the agent receives an observation o_t in the form of a screenshot of GUI running in a headless desktop environment, which it uses to update its internal state s_t before executing an action a_t under its current policy π .

about the UI’s usability strengths and weaknesses, then produces a numerical value (0-100) representing a usability score. This was to ensure the model provided a bounded assessment. We choose this method as over rubric-based scoring due to typical coverage deficiency and noisy evaluations from LLM-based scoring [69, 72, 74], and the fact that no comprehensive usability scale exists. This score was initialized by prompting the model to estimate the probability a user could successfully complete the agent’s tested flows given the observed design defects. The score was calibrated and further refined using a training objective that enforces a margin (i.e., minimum difference) between preferred and rejected UIs in our dataset.

4.2 Agent Architecture

Our CUAs are based on a popular open-source CUA agent harness [64] that captures a stream of screenshots from the user’s desktop, feeds them to the underlying model hosted on a vLLM inference endpoint [39], and uses generated responses to directly control the keyboard and mouse. This allows the CUA to run on any graphical desktop environment (e.g., operating system) and interact with any GUI regardless of the underlying implementation.

The CUA architecture follows the default settings of UI-TARS and maintains several data structures. First, a textual goal, G , provided to the agent at the beginning of execution. While most CUAs are trained to perform well-defined tasks (e.g., navigating to a web address), we use a static usability testing prompt (Appendix B) that instructs the model to make its own determination of important interaction flows to test for the purposes of rating usability. At each time step, the harness captures a visual screenshot, or observation o_t , which to update the agent’s internal state s_t , which consists of a window of the five most recent visual observations and a JSON-like representation of the session history. s_t , o_t , and a grounding instruction is used to prompt the the CUA model to reason about how to make progress towards G and describe observations (similar to a “think-aloud” exercise) to produce an executable action a_t . Valid actions are defined by the UI-TARS harness and exposed to the model through an initial system prompt. A single step of execution and processing lasts approximately five seconds. The agent stops its navigation when a special “stop” action is generated, e.g., if the model believes it has finished exploration, or if a maximum number of time steps (50) was reached. The final trace reflection and usability scoring is implemented by appending a manually-crafted evaluation prompt (Appendix B) as an additional turn in the session history and generating a response using the CUA. We provide an overview of this architecture in Figure 2.

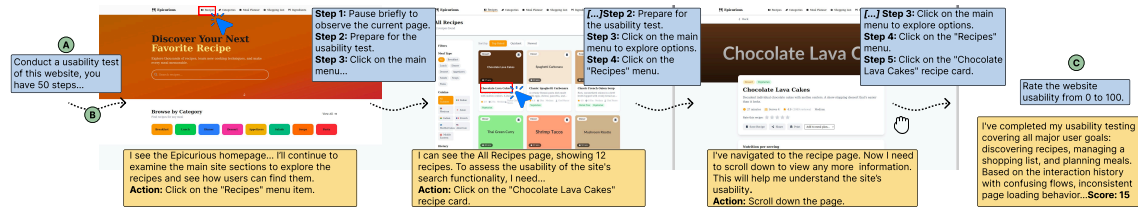


Fig. 3. A portion of an interaction trace that the model executes on a site from uxWeb. At the beginning (a) the model is asked to conduct a usability test; (b) the model starts exploration through an identified flow; (c) the model concludes its exploration and is prompted to assess the site's usability based on its interaction history.

4.3 Training Algorithm

We adopted an approach based on existing policy-based RL training pipelines [2, 81] to train the CUA to effectively *i)* explore UI flows, *ii)* test their functionality, and *iii)* assess their usability through a score. For our experiments, we chose the 8B or 9B versions of the best-performing open-source models on the OSWorld benchmark [87] as they were the strongest models that could fit on our hardware at the time of experiments.

In this section, we describe the three stages of our training approach: *i)* rollout generation, *ii)* reward assignment, and *iii)* policy update.

4.3.1 Rollout Generation. In policy-based reinforcement learning approaches, training data is obtained by running the current policy in the environment and collecting rollouts that reflect its present behavior, later adjusted through learning. In our experiments, we configured our agent harness with the base model (i.e., EvoCUA-8B, GUI-Owl-1.5-8B, Qwen3.5-9B) and generated up to 24 trace rollouts for each website in the training split of uxWeb by repeating its automated usability test protocol.

To perform rollout generation, we developed a containerized Apptainer environment that contained *i)* a headless X display server running at 1080p resolution, *ii)* an instance of Firefox running in full-screen mode, and *iii)* our agent harness which controlled a virtual mouse and keyboard. We wrote a script that repeatedly sampled a website from the training split at random, which was a React.js application bundle. This bundle was transferred to the container, where the website was staged and hosted at a pre-defined local URL. After successful staging, it was loaded into the browser, and a new usability testing session was started using our CUA harness. The CUA harness was configured to query a vLLM instance running on a compute cluster that served the base model. We configured our harness to log all interactions, model responses, and screenshots to accurately represent the states and actions of the rollout. Figure 3 shows a visualization of a portion of a collected rollout. We generated an average of 26,989 traces per model, taking approximately 2.5 weeks (machine hours) per model. Prompts for trace generation and scoring are provided in Appendix B.

4.3.2 Reward Assignment. After rollout generation, we evaluated and assigned rewards based on the *i)* quality of the navigation trace and *ii)* accuracy of the agent's predicted usability score. Rollouts that are assigned high reward scores are used to amplify effective behaviors within the model's policy.

Navigation Quality. The quality of navigation behavior has a large impact in usability assessment, and CUAs can often exhibit behaviors that are not characteristic of human users, such as failure to click graphical targets or "looping"

between a small number of screens [89]. We developed several heuristic measures to validate the navigation quality of rollouts used in training.

For each rollout, we computed the following metrics:

- **Same screen ratio:** The fraction of transitions with no visual change. Higher values indicate more repetitive behavior.
- **Same screen after clicks ratio:** The fraction of clicks that produced no screen change. Higher values suggest ineffective clicks or that the CUA became stuck.
- **Unique screen ratio:** The fraction of unique visited screens. Higher values indicate broader website exploration.
- **Steps:** The total number of steps in the interaction trace. Some traces were shorter due to early failure.

We filtered out blank screens (possibly due to load failures) before computing the first three metrics as they could artificially reduce these scores. We used perceptual hashing (phash) [38] with a Hamming distance of 4 to compute whether two screens were the same. Perceptual hashing has applications in reverse image search and identifying similar screens [1, 30, 37, 65]. We then combined these three metrics to compute the navigation quality score, s_{nav} :

$$s_{nav} = \text{Unique screen ratio} \times (1 - \text{Same screen ratio}) \times (1 - \text{Same screen after clicks ratio}) \quad (1)$$

We computed s_{nav} for unique screens in the rollout that were not separated by scrolling actions, which could trigger false positives in the screen hashing metric. We thresholded this score and rollout length to filter our generated rollouts. Through trial and error we determined that a s_{nav} of at least 0.07 and a minimum of 30 steps indicated a quality trace. We had an average of 3,973 traces per model that passed this quality check and used to train our models.

As an additional filter for *website* quality, we excluded websites in uxWeb if fewer than 3 of its 24 rollouts passed our heuristics, since we hypothesized that they were likely to contain implementation errors that prevented useful rollouts to be collected.

Usability Assessment Accuracy. After navigation, the CUA must accurately reflect on its observations, reason about the UI’s strengths and weaknesses, and provide a numerical usability score. Assigning a reward that reflects the accuracy of a rollout’s usability assessment is difficult due to the lack of ground truth for both the design reasoning and a usability score.

A straightforward but likely incorrect approach is to use a 100 as the target usability score for positive samples (plain websites) and 0 as the target usability score for negative samples (defect-augmented websites). Yet, the absolute quality of websites in our data is not known, e.g., usability defects may exist outside of the ones that were intentionally introduced, which would make 100 the incorrect target score.

To address this challenge, we introduce a method for dynamically estimating the target usability score of a website based on a model’s initial score predictions for both the website and its defect-augmented counterpart. Our approach, inspired by other contrastive learning objectives for text responses [40, 75] and visual design quality [85], enforces a margin between model-predicted scores of preferred and non-preferred samples (Figure 4). We first estimate the score of the plain website, μ_p , as the mean predicted usability score across all of its rollouts, and use the same procedure to estimate the score of the corresponding defect-augmented website, μ_d . If the estimated score difference does not satisfy the desired margin, i.e., $\mu_p - \mu_d < m$, we define adjusted target scores such that the positive website is assigned a target of $\mu_0 + \frac{m}{2}$ and the negative website is assigned a target of $\mu_0 - \frac{m}{2}$, which are the values closest to the initial model estimates that satisfy the margin and with $\mu_0 = \frac{\mu_p + \mu_d}{2}$. Reward values are then assigned to rollouts for each website

based on the proximity of their predicted usability scores to the corresponding target score, such that rollouts with predictions closer to the target receive higher reward. We chose the value of the margin m based on manual tuning, and we include it as a hyperparameter in the Appendix.

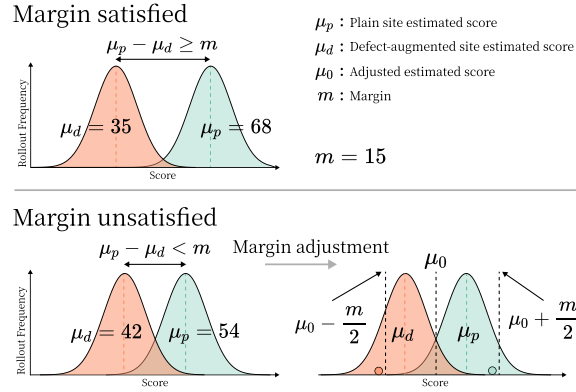


Fig. 4. If the estimated target scores for plain sites (μ_p) and defect-augmented sites (μ_d) satisfy the margin we use their estimated scores. Otherwise, we enforce a margin by shifting their estimated scores.

4.3.3 Policy Update. Finally, we used our dataset of reward-labeled rollouts to update the CUA’s policy model. For each site in the training split of uxWeb, we selected a single rollout (out of 24) with the highest assigned reward, which led to a total of 631 rollouts. We converted each rollout into multiple training examples using a sliding-window approach, where each training example contained the expected input and output format of the policy network at each timestep in the rollout [64, 89]. We generated an average of 21,228 windows per model for fine-tuning our policy. Examples from all websites in the uxWeb training split were aggregated for training. We used LLaMAFactory [97], an open source library, to conduct supervised fine-tuning using reward-labeled rollouts. We employed 4-bit quantization and LoRA to improve training efficiency. Hyperparameters were determined through manual experimentation and provided in Appendix D. Model fine-tuning was conducted on a GPU compute node with 4 H100 GPUs.

5 Technical Evaluations

We conducted two evaluations to assess models trained with our objective. Our first evaluation compared usability assessment accuracy, or the relative usability scoring, for models trained with our objective to that of existing open-source and proprietary CUA models. Our second evaluation assesses models’ intermediate usability reasoning during their navigation traces on synthetic and real websites through the quality of their usability critiques to see if more accurate assessments are accompanied with grounded observations.

5.1 Evaluation 1: Usability Assessments

We compared the accuracy of usability assessments of CUAs trained with our objective, CUAs trained with a naive objective, their open-source base models, and proprietary models. We evaluated the accuracy of model-predicted assessments using labels from uxWeb, which included both synthetic and human-provided labels.

5.1.1 *Baselines.* We used the following multimodal models as baselines for usability assessment:

- **GPT-5-mini** [56]: OpenAI’s proprietary efficient reasoning model for coding, computer use, and agentic workflows. GPT remains one of the leading choices for proprietary models and at the time of our experiments, this was one of the latest models released.
- **Kimi K2.5** [73]: Moonshot AI’s open-source 1.1T-parameter reasoning model for coding and computer use. At the time of our experiments, it ranked first among open-source models and second overall on OSWorld [87].
- **EvoCUA** [89]: Meituan’s open-source CUA model. Its 8B variant (EvoCUA-8B-20260105) was the strongest OSWorld open-source model that could run on consumer hardware at the time of our experiments.
- **GUI-Owl-1.5** [88]: Tongyi’s lab open-source CUA model. This model ranked second after EvoCUA on OSWorld. We selected its 8B variant.
- **Qwen3.5**: Alibaba’s most recent open-source model. At the time of our experiments, Qwen3.7 Plus ranked #3 on OSWorld but was not open-sourced. We selected this model as an alternative and its 9B variant.

5.1.2 *Model Training and Objectives.* We followed the training approach from §4.3 for each open-source model (EvoCUA, GUI-Owl-1.5, Qwen3.5). We trained each open-source model with our margin objective and performed an ablation with a naive 0/100 target objective, replacing the target usability scores of plain sites with 100 and defect-augmented sites with 0.

5.1.3 *Methodology.* We first constructed test samples, i.e., preference pairs, from the test split of uxWeb. From our test split, we randomly selected one plain clone per unique source website (30) and randomly sampled 4 of 8 of the plain clone’s associated defect-augmented variants for a total of 150 sites. We also collected all designer ratings of websites in the test split. In total, the benchmark consisted of 150 unique uxWeb website implementations and 115 comparison pairs.

During our evaluation, all CUAs were configured with the same harness and settings, which standardized their input/output modalities, i.e., visual observations of the GUI and keyboard/mouse interactions (§4.3.1), and generated chain-of-thought reasoning logs during their interactions. All conditions were run in our container-based GUI execution environment that staged websites locally and launched them in a full-screen Firefox browser. One interaction trace was generated for each evaluated site, limited to a maximum of 50 timesteps. Next, we applied each model to each site in a preference pair with the model’s prediction computed by taking the higher usability score of the sites in preference pairs. In cases where the model did not provide a score, we counted that as a vote for the other site.

We used **AUC** (area under precision-recall curve) as our primary metric to assess the degree to which the model’s predicted score discriminated between chosen sites (e.g., plain sites or human-preferred) over rejected ones (e.g., defect-augmented or human-rejected), compared to random chance. AUC accounts for the class imbalance in the four-to-one ratio of defect-augmented to plain websites in the test set.

5.1.4 *Results.* Overall, our training objective resulted in models with increased accuracy in usability assessments (Figure 5). Our margin objective lead to a consistent performance increase over base models (22.6%) compared to a naive 0/100 objective, which resulted in minimal improvement (8.0%). Additionally, our best fine-tuned model outperformed proprietary models by 18.1% on both splits.

Without training, *baseline models provide inaccurate usability assessments on UIs with known usability issues.* We observe how baseline models provide inaccurate usability assessments for defect-augmented sites (Figure 5), with

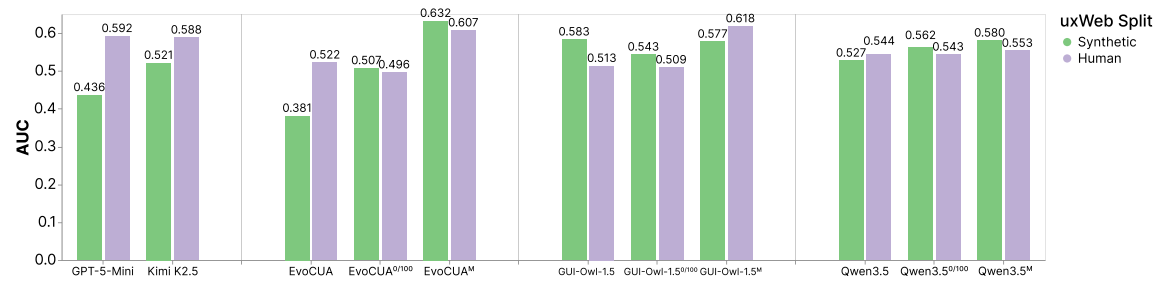


Fig. 5. Training with our margin objective leads to overall improvements over base models in AUC compared to a naive 0/100 objective. Our best performing model (EvoCUA^M) even outperforms proprietary models in providing accurate usability assessments for both test splits.

assessments worse than chance (GPT-5-mini, EvoCUA) or only slightly better than chance (Kimi K2.5, Qwen3.5, GUI-Owl-1.5). In contrast, our training objective results in models that can outperform their base models by 32.4% and proprietary models by 25.6%. Our best fine-tuned model demonstrates notable improvement in predicting usability assessments to sites with known usability issues, with an AUC of 0.632. This model provides a 37.0% increase in AUC over base and proprietary models, as well as a 65.9% increase over its base model (EvoCUA).

Incorporating training on human preference pairs with our objective also lead to overall improvement (12.8% increase) over base models. Similar to before, AUC for baseline models remains low (Figure 5). Their agreement rate with what humans label as more usable UIs suggests results comparable to random guessing. As described in §3.2, the original agreement rate between designers was low ($\alpha=0.308$) so large improvements may be difficult. Our best fine-tuned model achieves an AUC of 0.607.

Overall, our results suggest that our training paradigm with our data is an effective method for learning relative usability scoring, providing resulting models the ability to both form accurate usability assessments on sites with known defects and usability assessments aligned with human judgments.

5.2 Evaluation 2: Usability Critiques

Our second evaluation investigates whether the improved assessment capability resulting from our training objective was grounded in accurate observations from its model’s navigation trace. To do this, we examined the intermediate steps and reasoning generated by the model while the CUA performed its tasks and the final reflection phase used by the model before generating its final usability score. Since this requires manually inspection of traces, we focus this evaluation to the best model (EvoCUA^M) from §5.1 and its base model (EvoCUA).

5.2.1 Methodology. To evaluate the model’s internal critiques, we adopted the procedure from Duan et al. [14], which was used to evaluate the performance of a UI screenshot critic pipeline. We chose not to apply Nielsen and Molich [53]’s heuristic evaluation protocol since it primarily depends on overlap and recall of issues identified by human-experts, who do not “hallucinate” like models, i.e., requires measuring precision. Following existing protocol, a critique could be *i)* valid, *ii)* partially valid, or *iii)* invalid. We adapted the definitions to be applicable to interactions: a critique is considered **valid** if it exists on the site and was identified by the CUA during its navigation, **partially valid** if it does not exist on the site but resulted from the CUA’s trace (i.e. the CUA made a correct observation based on its trace but would not be an issue for humans such as misclicks due to time delay in harness which may have also resulted in the

incomplete execution of a user flow), and **invalid** if it does not exist and the CUA did not encounter the issue during its navigation trace (hallucination). Critiques were assigned 1.0, 0.5, and 0 points respectively. Three authors manually examined traces from 10 randomly selected plain benchmark sites of each model along with traces on corresponding real sites for a total of 20 sites. Authors were blinded to the model during annotation to reduce the chance that their ratings would be influenced by knowledge of the model that generated it. In total 251 issues were examined.

5.2.2 Results. Overall, we observed an average 23.1% increase in usability critique quality on real and synthetic sites. Table 1 shows the average usability critique quality, as calculated by the procedure introduced by Duan et al. [14]. We achieved similar inter-rater reliability score (Krippendorff’s $\alpha = 0.35$) as Duan et al. [14] (Fleiss’ $\kappa = 0.29$) on critique quality ratings.

Our fine-tuned model produced less partially valid critiques and less invalid critiques (Figure 6), suggesting a decrease in the number of hallucinated observations. Despite gains in both domains, the fine-tuned model had a smaller increase on real sites and lower overall performance. This suggests that our method is primarily effective in improving model performance on in-domain websites and usability defects but also leads to limited improvements on out-of-distribution data. On real-world data, the tuned model had a larger number of partially valid critiques. Upon manual inspection, we noticed that on real sites, models sometimes attempted to perform tasks that required login credentials then encountered verification issues that were interpreted by the model as usability faults. We marked these as partially valid, since they were faithful to the model’s real observations but would not constitute a problem for a human user. Overall, results from this experiment suggest that the better usability assessments previously observed are accompanied by more accurate grounding.

Table 1. Our training objective results in a model (EvoCUA^M) is also accompanied with overall better usability critique quality, or identified usability issues based on the model’s navigation trace, compared to its base model (EvoCUA).

Split	Model	Avg. Usability Critique Quality Score
Synthetic	EvoCUA	0.44
	EvoCUA ^M	0.62
Real	EvoCUA	0.37
	EvoCUA ^M	0.39

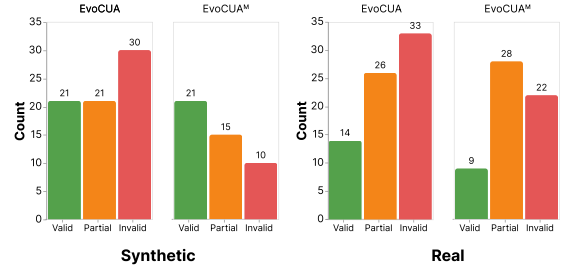


Fig. 6. Fine-tuned on our objective, EvoCUA^M overall produces less partially valid and invalid critiques on both synthetic and real websites, suggesting its increased usability assessment accuracy was grounded in its navigation traces.

6 Example Walkthrough

In this section, we provide an informal walkthrough³ as our model navigates a real-world site, demonstrating that it provides a usability assessment grounded in its interaction history.

We selected *craigslist.org* and provide an example walkthrough as our model navigates the website. We highlight areas where our model successfully and unsuccessfully explores the website and provide its final usability assessment as an example (Figure 7).

³For the purpose of this walkthrough we provide abridged versions of turns.

Our CUA begins when we instruct it to conduct a usability test on craigslist.org. The agent first waits for the page to load before beginning (1):

Agent: I'll start by identifying the most important user goals for this site. Craigslist is primarily used for posting and searching for classified ads, so I'll need to test these flows.

Observing the list of US states and cities, the agent decides to explore local listings in New York City (2), reasoning that is a major city that will enable it to better test the site's features. After navigating to New York City Craigslist page, the agent decides to test the functionality of the search box with a common item it believes to be most useful to users, a phone (3).

Agent: I see the page has loaded with various iPhone-related items. The search results are for "iphone" but the category filter on the left is set to "cell phones." I'll adjust the filter to "electronics" to refine results and test functionality.

Once the agent successfully refines its search (4), it decides to test flows for browsing and examining product details. To do so, the model then clicks on a result for an iPhone, spending a few turns clicking through the image gallery (5, 6). Once it finishes testing the gallery function, it progresses to see if it can view additional details about the phone or seller's contact information. After verifying that the seller's contact information can be viewed (7), it continues its usability test and attempts to navigate back to the main electronics listings page.

Here (8) the agent decides to click on the prev button at top of the screen to return to the main listing page, but ends up on a different iPhone listing instead. This is likely due to the fact that the way to navigate back to Craigslist's search results page is actually through the uparrow (↑) between the prev and next hyperlinks and not the hyperlinks themselves. At this point the agent, unable to navigate back to the listings page, decides to continue its current interaction flow by exploring more iPhone listings and examining each iPhone listing in detail to test the functionality of details views before it reaches the maximum number of steps (9, 10). At the end, the agent provides its final usability assessment:

Agent: I see issues with:

- **Navigation:** Repeatedly click on "prev" and "next" buttons to navigate content suggests inconsistent or unreliable navigation, indicating potential issues.
- **Confusing Flows:** Interaction history shows the assistant needs to scroll multiple times to view content, suggesting the layout or content organization may be unintuitive.
- **Failed Page Loads:** There were no page load failures, which is good.

Based on the these issues, the website usability score is 65.

Overall, we observe that the model is able to successfully navigate and identify key flows it should test on Craigslist, knowing how to search for items, identifying inconsistencies, and to explore page details. At the end of its exploration, the agent provides as grounded assessment based on its interaction history.

7 Discussion

We introduced a novel training objective to train CUAs to assess GUI usability and show that through our approach, we can outperform baselines and even beat proprietary models in accurate usability assessments, which is accompanied with more grounded usability assessments for real and synthetic UIs. We next discuss key results and broader implications.

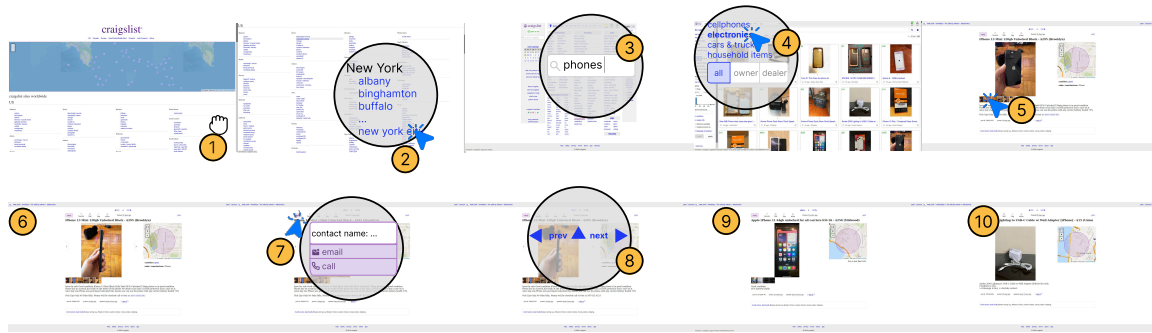


Fig. 7. An abridged interaction trace as the model conducts a usability test on craigslist.org. In this example, the agent chooses to explore flows related to searching for items on sale.

7.1 Computational Definition of Usability

A central goal of this work is to define usability assessment in a form that can be learned computationally. Usability depends on multiple factors, including aesthetics, functionality, and how easily users can discover important functionality. The UI perception and navigation capabilities of the underlying foundation model are important, but insufficient on their own. For example, our experiments found that models with stronger OSWorld navigation performance could navigate sites but lacked grounded usability assessments, frequently assigning websites with known defects usability scores akin to chance.

One way to operationalize usability is to prompt models with design guidelines [99] or expert-written critiques [14]. However, these approaches often produce invalid or only partially valid critiques, likely because positive examples alone do not discourage hallucinations and a model’s retrieved examples may come from mismatched contexts. Another approach is to supervise the evaluation process itself by training on expert traces. In practice, this is difficult: different evaluation methods can produce different findings [24]. Additionally traces are often noisy, and there is no single canonical path to a correct usability judgment. Such data is also hard to augment synthetically.

We therefore focused on outcome supervision rather than direct process supervision [43, 68]. Outcome labels are easier to generate at scale and less costly for humans to provide. Prior work has done this for static screenshots [82], but interactive usability assessment makes process and outcome tightly coupled: incorrect judgments may result from either poor exploration or poor reasoning. Our work is an initial step toward encoding an HCI evaluation procedure into a model, and we hope similar approaches can help formalize and learn other HCI methods.

7.2 From Better Models to New HCI Applications

Our approach builds on prior HCI work on user simulation for usability testing with the goal of enabling new, scalable HCI applications. To evaluate their final products, designers typically conduct usability studies where they provide participants with tasks [51], so simulating users means modeling how they respond to predefined stimuli or goals [3, 9, 93]. They then synthesize those results, weighing the severity and tradeoffs especially when comparing alternatives. As the web development cycle becomes increasingly rapid and agentic [32, 47], feedback must match the pace of development. We envision our method and resulting models as a way to provide designers early-stage evaluations before incorporating human-centered feedback as well as integrated into the automated design tools they use. We outline these applications below.

7.2.1 Scalable Simulated Usability Testing. Scalable simulated usability testing becomes increasingly necessary as the number of interfaces needing evaluation outpaces traditional usability testing. Our method can be used to train models that can provide these on-the-fly evaluations. Trained models can assess the usability of candidate interfaces and their alternatives, such as during A/B testing, generated by designers. These relative predictions can be repeatedly collected then aggregated before presented as a final score to designer or as a histogram of scores. As these assessments are made, usability critiques across runs can similarly be aggregated then grouped by issue type. Designers can then view the final score(s) and critiques for candidate interfaces to make judgments about the severity of the issues or to prioritize which issues to fix. They can fix flagged issues and improve the interface before the process is repeated, or the site is deployed/tested by real users. This enables rapid iteration before involving human feedback and may reduce usability issues in earlier stages of the development cycle.

7.2.2 Embedded Tool Call for Agents. Vibecoding platforms have seen a rise in popularity as they enable both designers and general users to create fully-interactive websites with a single prompt. Though current general purpose LLM services (e.g. Claude Artifacts [4], ChatGPT Canvas [55]) or specialized vibecoding platforms for website creation (e.g. Lovable [45], Replit [66]) have seen significant progress in website generation, the resulting sites are not necessarily usable. Often, they lack core functionality and beyond the first few webpages [13, 33] as they are primarily used in initial stages for planning. Similarly, coding agents (e.g. Codex, Claude Code) are also limited in assessing usability of generated UIs. Generated UIs may be visually pleasing but lack functionality [83] as agents fail to understand how visual adjustments impacts the UI’s usability [35]. We envision our method and resulting trained models integrated into these existing agents as an *agent tool*. After the agent (on the platform or coding agent) generates a UI, they can then perform an *agent tool call* to our models for usability assessment. Multiple usability assessments can be made for the agent to iteratively improve the interface before returning a final interface to the end-user.

7.2.3 Reward Signal for Improving UI Generation. We also envision that our method can be used to directly improve models that generate UIs and are integrated into current platform services or coding agents. Currently, generated UIs are limited to landing pages and small-scale sites with limited functionality [13, 33]. RLHF can be used to steer underlying models towards generating full-fledged, usable sites. However, existing RLHF methods used to improve generators ignore the rationale used to critique UI designs and improve their usability [84]. To enhance existing LLMs, a fine-tuned model with our method can first be used to assess generated UIs. The resulting relative usability assessment scores can be used by existing ranking-based RLHF methods to rank these UIs, enabling models to learn which UIs are more usable.

7.3 Limitations & Future Work

While we show our method can outperforms other baselines in our experimental data, we see several avenues for improvement.

7.3.1 Technical Limitations. Our CUAs are constrained by both compute limits and current CUA capabilities. Crawls were capped at 50 steps, likely preventing complete coverage of a website. This was due to practical constraints and by design: our training objective emphasized prioritizing important tasks rather than exhaustive exploration. Accordingly, unlike a traditional crawler, our CUAs were not intended to achieve full coverage in this initial work. Additionally, our CUAs are slow, requiring at least five second per step when accounting for the agent’s internal timing and model inference on our hardware. Due to this delay, they sometimes makes errors that humans typically would not (in §6),

such as missing click targets and misclassifying them as usability issues. However, our best performing CUA’s large improvements over its original model (EvoCUA) and performance compared to proprietary models still suggest that our training objective can be applied to more powerful models and improve their usability assessments. With less computational limits, future work can also explore applying our training approach with video streams to improve model usability assessment performance.

The model was trained primarily on fixed-resolution websites, due to the available synthetic data and to the training environment of the base model [89]. We expect this approach to transfer to similar 2D WIMP interfaces, but likely less well to mobile settings. Finally, our website evaluations did not include strong guardrails against high-impact actions, such as making purchases or deleting data. Prior work highlights the importance of accounting for the consequences of risky or irreversible UI actions [95], which was one reason we limited evaluation on real-world sites.

7.3.2 Generalizability. Our CUAs were trained and primarily evaluated on synthetic websites, limiting its generalizability. We hypothesize that with access to real world interface variants, such as alternative UIs produced during A/B testing, or datasets collected by large scale LLM benchmarks (e.g. LM Arena), model performance would improve in these domains as our evaluations show overall in-domain improvements. We used synthetic sites because they allow controlled manipulation of usability characteristics, reproducible evaluation, and terms-of-service-compliant data collection without requiring user accounts or payment information. However, they are less polished than real websites and may not capture the full range of real-world usability issues. Prior work has shown that CUAs can be influenced by dark patterns, suggesting that some interfaces may be harder for both our model and humans to evaluate [27]. Although our walkthrough provides some evidence that our CUAs can operate on real websites, our findings may not extend to all intended use cases, such as other other GUI types (e.g., native desktop or mobile apps) or intermediate stages of a designer’s workflow. Still, the strong gains over EvoCUA-8B on both splits of the synthetic dataset suggest that our architecture and training approach are effective when suitable data is available, and may transfer to other domains with sufficient domain-specific data.

7.3.3 Contextualized Evaluation. Finally, we note limitations of our evaluation and directions for future work. We focused on outcome decision and critique accuracy, but these do not fully capture the process by which our CUAs reaches its assessments. Even when the model matches human preferences, it is unclear whether this is because it explores the same parts of the interface or relies on similarly meaningful evidence.

Different use cases will also require different capabilities and evaluation criteria. Our approach focused on training CUAs to provide accurate usability scores as a relative assessment for a site’s usability, in other scenarios such as designer judging, direct designer agreement may be more appropriate. Similarly, in other contexts, designers may value actionable feedback and control over the tasks, instructions, and context provided to the model. Practitioners using our CUAs in model training may need interpretable and calibrated scores. In end-user or generative UI settings, the model may also need to adapt to individual preferences and accessibility needs. We hope our released dataset and models provide a foundation for these more targeted applications and evaluations.

8 Conclusion

We present a novel training objective to train CUAs to assess GUI usability. We used it to fine-tune different open-source base models. To support this goal, we created uxWeb, a large-scale dataset of interactive websites annotated with both synthetic defect-based usability labels and human judgments. Across two evaluations, our method results in consistent gains over base models, enables our best-performing fine-tuned model to outperform larger proprietary models, and is

accompanied with grounded usability critiques on real and synthetic interfaces. These findings suggest that current off-the-shelf agents are not (yet) sufficient for rigorous usability assessment, but that usability assessment can be learned when paired with appropriate data and objectives. More broadly, we hope this work provides a foundation for scalable, reproducible, and human-aligned usability evaluation in HCI, and that releasing our data and models will support future research on automated design evaluation, benchmarking, and interactive AI systems.

References

- [1] Pieter Agten, Wouter Joosen, Frank Piessens, and Nick Nikiforakis. 2015. Seven months’ worth of mistakes: A longitudinal study of typosquatting abuse. In *Proceedings of the 22nd Network and Distributed System Security Symposium (NDSS 2015)*. Internet Society.
- [2] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. arXiv:2402.14740 [cs.LG] <https://arxiv.org/abs/2402.14740>
- [3] John R. Anderson and Christian Lebiere. 1998. *The Atomic Components of Thought*. Lawrence Erlbaum Associates, Mahwah, NJ.
- [4] Anthropic. 2025. Turn ideas into interactive AI-powered apps. (2025). <https://www.anthropic.com/news/build-artifacts>
- [5] Anthropic. 2026. Claude Code: Overview (Documentation). <https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview>. Accessed 2026-02-06.
- [6] Apple. 2023. Human Interface Guidelines. <https://developer.apple.com/design/human-interface-guidelines>
- [7] Arcada Labs. 2026. DesignArena by Arcada Labs. <https://www.designarena.ai/>. Accessed 2026-02-06.
- [8] Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. 2024. DigiRL: Training In-The-Wild Device-Control Agents with Autonomous Reinforcement Learning. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. doi:10.52202/079017-0397
- [9] Stuart Card, THOMASP MORAN, and Allen Newell. 1986. The model human processor- An engineering model of human performance. *Handbook of perception and human performance*. 2, 45–1 (1986), 1–35.
- [10] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems*. https://proceedings.neurips.cc/paper_files/paper/2023/file/5950bf290a1570ea401bf98882128160-Paper-Datasets_and_Benchmarks.pdf
- [11] Linda Di Geronimo, Larissa Braz, Enrico Fregnan, Fabio Palomba, and Alberto Bacchelli. 2020. UI Dark Patterns and Where to Find Them: A Study on Mobile Applications and User Perception. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI ’20)*. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/3313831.3376600
- [12] Alan Dix. 2004. *Human-computer interaction*. Vol. 1. Pearson Education.
- [13] Dolfia. 2026. Why AI is exposing design’s craft crisis. <https://uxdesign.cc/why-ai-is-exposing-designs-craft-crisis-434bcb652848>. Accessed 2026-09-02.
- [14] Peitong Duan, Chin-Yi Cheng, Gang Li, Bjoern Hartmann, and Yang Li. 2024. UICrit: Enhancing Automated Design Evaluation with a UI Critique Dataset. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (Pittsburgh, PA, USA) (UIST ’24)*. Association for Computing Machinery, New York, NY, USA, Article 46, 17 pages. doi:10.1145/3654777.3676381
- [15] Leah Findlater and Joanna McGrenere. 2007. Evaluating reduced-functionality interfaces according to feature findability and awareness. In *IFIP Conference on Human-Computer Interaction*. Springer, 592–605.
- [16] Paul M Fitts. 1954. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of experimental psychology* 47, 6 (1954), 381.
- [17] Raymond Fok, Mingyuan Zhong, Anne Spencer Ross, James Fogarty, and Jacob O. Wobbrock. 2022. A Large-Scale Longitudinal Analysis of Missing Label Accessibility Failures in Android Apps. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (New Orleans, LA, USA) (CHI ’22)*. Association for Computing Machinery, New York, NY, USA, Article 461, 16 pages. doi:10.1145/3491102.3502143
- [18] Hiroki Furuta, Kuang-Huei Lee, Ofir Nachum, Yutaka Matsuo, Aleksandra Faust, Shixiang Shane Gu, and Izzeddin Gur. 2023. Multimodal Web Navigation with Instruction-Finetuned Foundation Models. arXiv:2305.11854 [cs.AI]
- [19] Hiroki Furuta, Kuang-Huei Lee, Ofir Nachum, Yutaka Matsuo, Aleksandra Faust, Shixiang Shane Gu, and Izzeddin Gur. 2024. Multimodal Web Navigation with Instruction-Finetuned Foundation Models. arXiv:2305.11854 [cs.LG] <https://arxiv.org/abs/2305.11854>
- [20] Krzysztof Gajos and Daniel S Weld. 2004. SUPPLE: automatically generating user interfaces. In *Proceedings of the 9th international conference on Intelligent user interfaces*. 93–100.
- [21] Krzysztof Gajos and Daniel S Weld. 2005. Preference elicitation for interface optimization. In *Proceedings of the 18th annual ACM symposium on User interface software and technology*. 173–182.
- [22] Yifei Gao, Jiang Wu, Xiaoyi Chen, Yifan Yang, Zhe Cui, Tianyi Ma, Jiaming Zhang, and Jitao Sang. 2026. GUITester: Enabling GUI Agents for Exploratory Defect Discovery. *arXiv preprint arXiv:2601.04500* (2026). doi:10.48550/arXiv.2601.04500
- [23] Kelley Gordon. 2020. 5 Principles of Visual Design in UX. <https://www.nngroup.com/articles/principles-visual-design/>
- [24] Wayne D. Gray and Marilyn C. Salzman. 1998. Damaged Merchandise? A Review of Experiments That Compare Usability Evaluation Methods. *Human-Computer Interaction* 13, 3 (1998), 203–261. doi:10.1207/S15327051HCI1303_2

- [25] George Gui and Olivier Toubia. 2023. The Challenge of Using LLMs to Simulate Human Behavior: A Causal Inference Perspective. (2023). doi:10.2139/ssrn.4650172
- [26] Johanna Gunawan, Amogh Pradeep, David Choffnes, Woodrow Hartzog, and Christo Wilson. 2021. A Comparative Study of Dark Patterns Across Web and Mobile Modalities. *Proc. ACM Hum.-Comput. Interact.* 5, CSCW2, Article 377 (Oct. 2021), 29 pages. doi:10.1145/3479521
- [27] Longjie Guo, Chenjie Yuan, Mingyuan Zhong, Robert Wolfe, Ruican Zhong, Yue Xu, Bingbing Wen, Hua Shen, Lucy Lu Wang, and Alexis Hiniker. 2026. SusBench: An Online Benchmark for Evaluating Dark Pattern Susceptibility of Computer-Use Agents. In *Proceedings of the 31st International Conference on Intelligent User Interfaces (IUI '26)*. Association for Computing Machinery, New York, NY, USA, 1917–1937. doi:10.1145/3742413.3789111
- [28] Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2024. A Real-World WebAgent with Planning, Long Context Understanding, and Program Synthesis. arXiv:2307.12856 [cs.LG] <https://arxiv.org/abs/2307.12856>
- [29] Perttu Hämäläinen, Mikke Tavast, and Anton Kunnari. 2023. Evaluating Large Language Models in Generating Synthetic HCI Research Data: a Case Study. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3544548.3580688
- [30] Qingying Hao, Licheng Luo, Steve T.K. Jan, and Gang Wang. 2021. It's Not What It Looks Like: Manipulating Perceptual Hashing based Applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, Republic of Korea) (CCS '21)*. Association for Computing Machinery, New York, NY, USA, 69–85. doi:10.1145/3460120.3484559
- [31] Aurora Harley. 2018. UX Expert Reviews. <https://www.nngroup.com/articles/ux-expert-reviews/>.
- [32] Steffen Holter, Eunye Koh, Mustafa Doga Dogan, and Gromit Yeuk-Yin Chan. 2026. UXCascade: Scalable Usability Testing with Simulated User Agents. arXiv:2601.15777 [cs.HC] <https://arxiv.org/abs/2601.15777>
- [33] Igor Kraisnik. 2026. Why AI-Generated UI Fails in Production When Prototypes Become Real Products. <https://ministryofprogramming.com/blog/why-ai-generated-ui-fails-in-production>. Accessed 2026-09-02.
- [34] Aleks Ikkala, Florian Fischer, Markus Klar, Miroslav Bachinski, Arthur Fleig, Andrew Howes, Perttu Hämäläinen, Jörg Müller, Roderick Murray-Smith, and Antti Oulasvirta. 2022. Breathing Life Into Biomechanical User Models. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (Bend, OR, USA) (UIST '22)*. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/3526113.3545689
- [35] Jaehyun Jeon, Min Soo Kim, Janghan Yoon, Sumin Shim, Yejin Choi, Hanbin Kim, Dae Hyun Kim, and Youngjae Yu. 2026. Do MLLMs Capture How Interfaces Guide User Behavior? A Benchmark for Multimodal UI/UX Design Understanding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (Eds.). Association for Computational Linguistics, San Diego, California, United States, 44269–44294. doi:10.18653/v1/2026.acl-long.2049
- [36] Shivani Kapania, William Agnew, Motahhare Eslami, Hoda Heidari, and Sarah E. Fox. 2025. Simulacrum of Stories: Examining Large Language Models as Qualitative Research Participants. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3706598.3713220 Honorable Mention.
- [37] Panagiotis Kintis, Najmeh Miramirkhani, Charles Lever, Yizheng Chen, Rosa Romero-Gómez, Nikolaos Pitropakis, Nick Nikiforakis, and Manos Antonakakis. 2017. Hiding in plain sight: A longitudinal study of combosquatting abuse. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 569–586.
- [38] Evan Klinger and David Starkweather. 2021. pHash: The open source perceptual hash library. <https://www.phash.org/docs/>
- [39] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [40] Nathan Lambert. 2026. *The RLHF Book: Reinforcement Learning from Human Feedback, Alignment, and Post-Training LLMs*. Manning Publications. <https://www.manning.com/books/the-rlhf-book> Manning Early Access Program (MEAP); publication estimated Summer 2026; last updated January 2026.
- [41] Chunyang Li, Yilun Zheng, Xinting Huang, Tianqing Fang, Jiahao Xu, Lihui Chen, Yangqiu Song, and Han Hu. 2026. WebDevJudge: Evaluating (M)LLMs as Critiques for Web Development Quality. arXiv:2510.18560 [cs.SE] <https://arxiv.org/abs/2510.18560>
- [42] William Lidwell, Kritina Holden, and Jill Butler. 2010. *Universal principles of design, revised and updated: 125 ways to enhance usability, influence perception, increase appeal, make better design decisions, and teach through design*. Rockport Pub.
- [43] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050* (2023). arXiv:2305.20050 [cs.LG] doi:10.48550/arXiv.2305.20050
- [44] LMArena Team. 2025. WebDev Arena: A Live LLM Leaderboard for Web App Development. <https://lmarena.ai/blog/webdev-arena/>. Published 2025-03-09; contributors: Aryan Vichare, Anastasios N. Angelopoulos, Wei-Lin Chiang, Kelly Tang, Luca Manolache; accessed 2026-02-06.
- [45] Lovable. 2025. Lovable. (2025). <https://lovable.dev/>
- [46] Yuxuan Lu, Jing Huang, Yan Han, Bingsheng Yao, Sisong Bei, Jiri Gesi, Yaochen Xie, Yisi Sang, Zheshen, Wang, Qi He, and Dakuo Wang. 2026. Can LLM Agents Simulate Multi-Turn Human Behavior? Evidence from Real Online Customer Behavior Data. arXiv:2503.20749 [cs.CL] <https://arxiv.org/abs/2503.20749>
- [47] Yuxuan Lu, Bingsheng Yao, Hansu Gu, Jing Huang, Zheshen Jessie Wang, Yang Li, Jiri Gesi, Qi He, Toby Jia-Jun Li, and Dakuo Wang. 2025. UXAgent: An LLM Agent-Based Usability Testing Framework for Web Design. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*. Association for Computing Machinery, New York, NY, USA, Article 545, 12 pages. doi:10.1145/3706599.3719729

- [48] Kurt Luther, Jari-Lee Tolentino, Wei Wu, Amy Pavel, Brian P. Bailey, Maneesh Agrawala, Björn Hartmann, and Steven P. Dow. 2015. Structuring, Aggregating, and Evaluating Crowdsourced Design Critique. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing* (Vancouver, BC, Canada) (CSCW '15). Association for Computing Machinery, New York, NY, USA, 473–485. doi:10.1145/2675133.2675283
- [49] Thomas Mahatody, Mouldi Sagar, and Christophe Kolski. 2010. State of the art on the cognitive walkthrough method, its variants and evolutions. *Intl. Journal of Human-Computer Interaction* 26, 8 (2010), 741–785.
- [50] Hee Seung Moon, Antti Oulasvirta, and Byungjoo Lee. 2023. Amortized Inference with User Simulations. In *CHI 2023 - Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM. doi:10.1145/3544548.3581439
- [51] Jakob Nielsen. 2012. Usability 101: Introduction to Usability. <https://www.nngroup.com/articles/usability-101-introduction-to-usability/>.
- [52] Jakob Nielsen and Thomas K. Landauer. 1993. A mathematical model of the finding of usability problems. In *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems* (Amsterdam, The Netherlands) (CHI '93). Association for Computing Machinery, New York, NY, USA, 206–213. doi:10.1145/169059.169166
- [53] Jakob Nielsen and Rolf Molich. 1990. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Seattle, Washington, USA) (CHI '90). Association for Computing Machinery, New York, NY, USA, 249–256. doi:10.1145/97243.97281
- [54] OpenAI. 2023. GPT-4V(ision) System Card. https://cdn.openai.com/papers/GPTV_System_Card.pdf. OpenAI system card.
- [55] OpenAI. 2024. Introducing canvas. (2024). <https://openai.com/index/introducing-canvas/>
- [56] OpenAI. 2025. Introducing GPT-5. <https://openai.com/index/introducing-gpt-5/>
- [57] Antti Oulasvirta, Samuli De Pascale, Janin Koch, Thomas Langerak, Jussi Jokinen, Kashyap Todi, Markku Laine, Manoj Krishthombuge, Yuxi Zhu, Aliaksei Miniukovich, et al. 2018. Aalto interface metrics (AIM) a service and codebase for computational GUI evaluation. In *Adjunct proceedings of the 31st annual ACM symposium on user interface software and technology*. 16–19.
- [58] Antti Oulasvirta, Jussi P. Jokinen, and Andrew Howes. 2022. Computational rationality as a theory of interaction. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–14.
- [59] Joon Sung Park, Lindsay Popowski, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2022. Social Simulacra: Creating Populated Prototypes for Social Computing Systems. arXiv:2208.04024 [cs.HC] <https://arxiv.org/abs/2208.04024>
- [60] Fabio Paternò, Cristiano Mancini, and Silvia Meniconi. 1997. ConcurTaskTrees: A Diagrammatic Notation for Specifying Task Models. In *Human-Computer Interaction INTERACT '97*, Steve Howard, Judy Hammond, and Gitte Lindgaard (Eds.). Springer, Boston, MA, 362–369. doi:10.1007/978-0-387-35175-9_58
- [61] Yi-Hao Peng, Jeffrey P. Bigham, and Jason Wu. 2025. DesignPref: Capturing Personal Preferences in Visual Design Generation. arXiv:2511.20513 [cs.CV] <https://arxiv.org/abs/2511.20513>
- [62] Angel R. Puerta. 1997. A Model-Based Interface Development Environment. *IEEE Software* 14, 4 (1997), 40–47.
- [63] Zehan Qi, Xiao Liu, Iat Long Long, Hanyu Lai, Xueqiao Sun, Jiadai Sun, Xinyue Yang, Yu Yang, Shuntian Yao, Wei Xu, Jie Tang, and Yuxiao Dong. 2025. WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.
- [64] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv:2501.12326 [cs.AI] <https://arxiv.org/abs/2501.12326>
- [65] Jathushan Rajasegaran, Naveen Karunanayake, Ashanie Gunathillake, Suranga Seneviratne, and Guillaume Jourjon. 2019. A multi-modal neural embeddings approach for detecting mobile counterfeit apps. In *The World Wide Web Conference*. 3165–3171.
- [66] Replit. 2025. Replit – Build apps and sites with AI. (2025). <https://replit.com/>
- [67] Shreya Shankar, JD Zamfirescu-Pereira, Björn Hartmann, Aditya Parameswaran, and Ian Arawjo. 2024. Who validates the validators? aligning llm-assisted evaluation of llm outputs with human preferences. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–14.
- [68] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300 (2024). arXiv:2402.03300 [cs.CL] doi:10.48550/arXiv.2402.03300
- [69] William F. Shen, Xinchu Qiu, Chenxi Whitehouse, Lisa Alazraki, Shashwat Goel, Francesco Barbieri, Timon Willi, Akhil Mathur, and Ilias Leontiadis. 2026. Rethinking Rubric Generation for Improving LLM Judge and Reward Modeling for Open-ended Tasks. arXiv:2602.05125 [cs.LG] <https://arxiv.org/abs/2602.05125>
- [70] Danqing Shi, Yujun Zhu, Jussi P. P. Jokinen, Aditya Acharya, Aini Putkonen, Shumin Zhai, and Antti Oulasvirta. 2024. CRTypist: Simulating Touchscreen Typing Behavior via Computational Rationality. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery. doi:10.1145/3613904.3642918
- [71] Ben Shneiderman. 2010. *Designing the user interface: strategies for effective human-computer interaction*. Pearson Education India.
- [72] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2024. Judgebench: A benchmark for evaluating llm-based judges. arXiv preprint arXiv:2410.12784 (2024).
- [73] Kimi Team. 2026. Kimi K2.5: Visual Agentic Intelligence. arXiv:2602.02276 [cs.CL] <https://arxiv.org/abs/2602.02276>

- [74] Aman Singh Thakur, Kartik Choudhary, Venkat Srinik Ramayapally, Sankaran Vaidyanathan, and Dieuwke Hupkes. 2025. Judging the judges: Evaluating alignment and vulnerabilities in llms-as-judges. In *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM²)*. 404–430.
- [75] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. (2023). arXiv:2307.09288 [cs.CL] doi:10.48550/arXiv.2307.09288
- [76] Daniel Toyama, Philippe Hamel, Anita Gergely, Gheorghe Comanici, Amelia Glaese, Zafarali Ahmed, Tyler Jackson, Shibl Mourad, and Doina Precup. 2021. AndroidEnv: A Reinforcement Learning Platform for Android. arXiv:2105.13231 [cs.LG] doi:10.48550/arXiv.2105.13231
- [77] Browser Use. 2026. Browser Use: The AI browser agent. <https://browser-use.com>
- [78] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling conversational interaction with mobile ui using large language models. In *Proceedings of the 2023 CHI conference on human factors in computing systems*. 1–17.
- [79] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, et al. 2025. Opencua: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123* (2025).
- [80] Zichao Wang and Alexa Siu. 2026. Interview-Informed Generative Agents for Product Discovery: A Validation Study. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM. Honorable Mention.
- [81] Ronald J Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning* 8, 3 (1992), 229–256.
- [82] Jason Wu, Yi-Hao Peng, Amanda Li, Amanda Swearngin, Jeffrey P. Bigham, and Jeffrey Nichols. 2024. UIClip: A Data-driven Model for Assessing User Interface Design. arXiv:2404.12500 [cs.HC] <https://arxiv.org/abs/2404.12500>
- [83] Jason Wu, Eldon Schoop, Alan Leung, Titus Barik, Jeffrey P. Bigham, and Jeffrey Nichols. 2024. UICoder: Finetuning Large Language Models to Generate User Interface Code through Automated Feedback. arXiv:2406.07739 [cs.CL] <https://arxiv.org/abs/2406.07739>
- [84] Jason Wu, Amanda Swearngin, Arun Krishnavajjala, Alan Leung, Jeffrey Nichols, and Titus Barik. 2026. Improving User Interface Generation Models from Designer Feedback. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. ACM, 1–17. doi:10.1145/3772318.3791567
- [85] Jason Wu, Amanda Swearngin, Arun Krishna Vajjala, Alan Leung, Jeffrey Nichols, and Titus Barik. 2025. Improving User Interface Generation Models from Designer Feedback. (Sept. 2025). arXiv:2509.16779 [cs.HC] doi:10.48550/arXiv.2509.16779
- [86] Wei Xiang, Hanfei Zhu, Suqi Lou, Xinli Chen, Zhenghua Pan, Yuxiao Jin, Shuyue Chen, and Lingyun Sun. 2024. SimUser: Generating Usability Feedback by Simulating Various Users Interacting with Mobile Applications. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA, 1–17. doi:10.1145/3613904.3642481
- [87] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-1650
- [88] Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, Zhiyuan Chen, Jitong Liao, Qi Zheng, Jiahui Zeng, Ze Xu, Shuai Bai, Junyang Lin, Jingren Zhou, and Ming Yan. 2026. Mobile-Agent-v3.5: Multi-platform Fundamental GUI Agents. arXiv:2602.16855 [cs.AI] <https://arxiv.org/abs/2602.16855>
- [89] Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, Jinrui Ding, Xiandi Ma, Yuchen Xie, Peng Pei, Xunliang Cai, and Xipeng Qiu. 2026. EvoCUA: Evolving Computer Use Agents via Learning from Scalable Synthetic Experience. arXiv:2601.15876 [cs.AI] <https://arxiv.org/abs/2601.15876>
- [90] Bo Yang, Zhenchang Xing, Xin Xia, Chunyang Chen, Deheng Ye, and Shanping Li. 2021. Don't do that! hunting down visual design smells in complex uis against design guidelines. In *2021 IEEE/ACM 43rd international conference on software engineering (ICSE)*. IEEE, 761–772.
- [91] Shengcheng Yu, Chunrong Fang, Ziyuan Tuo, Qunjun Zhang, Chunyang Chen, Zhenyu Chen, and Zhendong Su. 2025. Vision-Based Mobile App GUI Testing: A Survey. *Comput. Surveys* 58, 6, Article 142 (Dec. 2025), 46 pages. doi:10.1145/3773027
- [92] zai-org. 2025. GLM-4.7. <https://huggingface.co/zai-org/GLM-4.7>. Hugging Face model card; accessed 2026-02-06.
- [93] Chenchen Zhang, Yuhang Li, Can Xu, Jiaheng Liu, Ao Liu, Changzhi Zhou, Ken Deng, Dengpeng Wu, Guanhua Huang, Kejiao Li, Qi Yi, Ruibin Xiong, Shihui Hu, Yue Zhang, Yuhao Jiang, Zenan Xu, Yuanxing Zhang, Wiggan Zhou, Chayse Zhou, and Fengzong Lian. 2025. ArtifactsBench: Bridging the Visual-Interactive Gap in LLM Code Generation Evaluation. *arXiv preprint arXiv:2507.04952* (2025). doi:10.48550/arXiv.2507.04952
- [94] Zhuohao (Jerry) Zhang, Ruiqi Chen, Mingyuan Zhong, and Jacob O. Wobbrock. 2025. SlideAudit: A Dataset and Taxonomy for Automated Evaluation of Presentation Slides. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*. Association for Computing Machinery, New York, NY, USA, Article 13, 23 pages. doi:10.1145/3746059.3747736

- [95] Zhuohao (Jerry) Zhang, Eldon Schoop, Jeffrey Nichols, Anuj Mahajan, and Amanda Swearngin. 2025. From Interaction to Impact: Towards Safer AI Agents Through Understanding and Evaluating Mobile UI Operation Impacts. In *Proceedings of the 30th International Conference on Intelligent User Interfaces*. ACM, 727–744. doi:10.1145/3708359.3712153
- [96] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
- [97] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyang Luo. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Yixin Cao, Yang Feng, and Deyi Xiong (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 400–410. doi:10.18653/v1/2024.acl-demos.38
- [98] Mingyuan Zhong, Ruolin Chen, Xia Chen, James Fogarty, and Jacob O Wobbrock. 2025. Screenaudit: Detecting screen reader accessibility errors in mobile apps using large language models. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–19.
- [99] Ruican Zhong, David W. McDonald, and Gary Hsieh. 2025. Synthetic Heuristic Evaluation: A Comparison between AI- and Human-Powered Usability Evaluation. arXiv:2507.02306 [cs.HC] <https://arxiv.org/abs/2507.02306>

A uxWeb Creation Prompts

A.1 Synthetic Website Generation

We used the following prompt for GLM 4.7 when generating websites clones of the 137 real-world sites from Mind2Web. This prompt was determined by trial and error and manual inspection on a small number of test cases.

You are an expert web developer and designer specializing in modern websites. You are currently handed a barebones React.js template and your jobs is to make a fully working website using React.js. Feel free to use lightweight libraries like Tailwind CSS to enhance the design.

Requirements:

1. Create a fully functional, modern, and responsive website design
2. Use only React.js, HTML, CSS, and JavaScript, but feel free to use libraries like Tailwind CSS to make the design better and make layout easier. You can look at the package.json file to see which libraries are already installed and import them. However, you must NEVER install new packages that were not already installed.
3. Include interactive elements where appropriate
4. Make it production-ready and professional
5. You must include all relevant script tags for libraries to work properly.
6. There should NEVER be any dead links or buttons that are visible but do nothing when clicked.
7. The ONLY time you are allowed to run the `npm` command is to test if the website builds with `npm run build` command. You must NEVER run `npm` under any other circumstances.
8. Because you will create a website with very realistic mocked functionality, you will probably need some data. ALWAYS synthetically generate placeholder data that is needed by any functionality on the mocked website. NEVER attempt to connect to online APIs.
9. NEVER include images for which you do not have the appropriate path to (e.g., the "src" attribute). You should use an API to generate a placeholder instead like ``
10. Do not define any new SVGs - you should use icons from FontAwesome instead from the packages: `@fortawesome/fontawesome-svg-core` `@fortawesome/free-solid-svg-icons` `@fortawesome/react-fontawesome`. You can assume that these packages are installed even though you cannot see them in a package.json in the same directory.
11. NEVER attempt to access any external directory outside of your current directory.

Your job is to create a fully functioning clone of the following website: `<WEBSITE_NAME>`

Use the current barebones React.js project with React Router to implement this website, with all important functionality realistically mocked. The main React file is located at `src/App.jsx` and exports a component called "App." Make sure your code conforms to this structure. Your workflow should be something like the following:

1. Think about the target website. Think about as many flows as you can that a user would want to do on this website.
2. Implement the React.js website so that all of flows you thought are fully implemented, interactive, and using the appropriate mock data.
3. Run ``npm run build`` command to test that website compiles without any errors.
4. Return to step 1 and repeat until you cannot think of any more flows.

When you think you are done, scan for any dead links or unimplemented functionality and implement them.

Finally, when you are done, make a txt file in the root directory of the React.js project (at the same level as the `src/` directory) called "flows.txt" that contains all of the flows that you implemented.

A.2 Usability Defect Injection Prompt

We used the following prompt to inject usability defects for our defect-augmented websites. For each website clone, we used this prompt up to eight times to inject usability defects aligned to one of our principles with the provided definition in §3.1.2.

You are an expert web developer and UI/UX designer in charge of usability testing. You will be given a completely 1) a completely functional website and 2) potential user flows that can be performed on the website. The website is implemented with React.js with React Router. Your job is to introduce code changes that violate a specific interaction design principle, making the app less usable for that principle, while not breaking the website.

Types of changes you can make:

1. Delete: // deleting UI elements
2. Add: // including additional UI elements
3. Modify: // modifying UI elements and their functionality

App: <WEBSITE_NAME>

Flows: flows.txt

Principle: <PRINCIPLE>

Using the current React.js code in the `src/` directory:

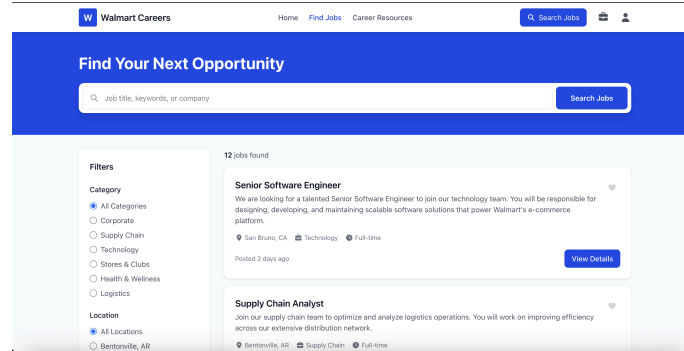
1. Consider all user flows inside the .txt file and all, the critical steps to these flows, and what changes can be introduced to violate the given principle
2. Change the code in the current directory to violate the given principle. Ensure these changes are fully implemented, interactive, and use the appropriate mock data or modified mock data
3. Run ``npm run build`` command to test that website compiles without any errors.
4. Run ``npm run build`` to check if all changes made in step 2 are implemented AND show up in the modified code

1249 When you are done, make a text file in the current directory (src/) called changes.txt listing
1250 all the changes made to the application that have made it more difficult to use.

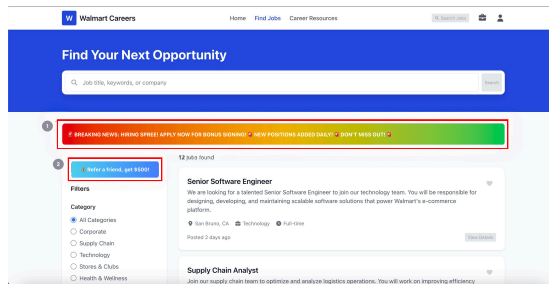
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300

A.3 Defect-Augmented Examples

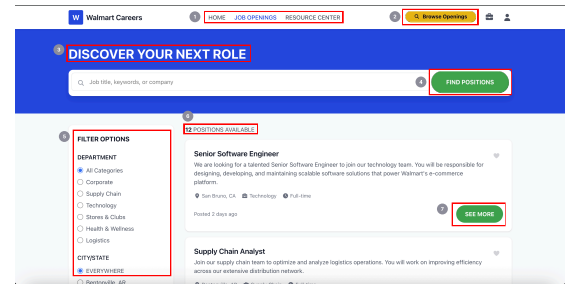
We show four example defect-augmented sites and the usability principle they violate in Figure 8.



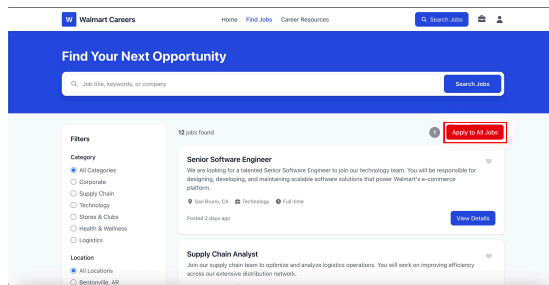
(a) Baseline Interface (Plain synthetic clone)



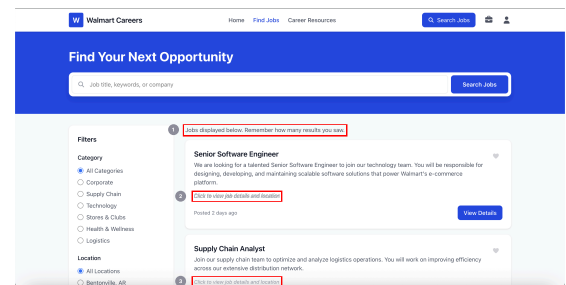
(b) Hierarchy



(c) Consistency



(d) Error Prevention



(e) Memory Load

Fig. 8. Comparison of an original clone from uxWeb and four of its defect-augmented counterparts. (a) shows the baseline design of a clone of the Walmart Careers job search portal. (b)-(e) illustrate different usability defects introduced to create our defect-augmented sites.

B Trace Generation Prompts

We used the following prompt when generating interaction traces on uxWeb:

Please generate the next move according to the UI screenshot, instruction and previous actions.

Instruction:
Conduct a usability test of this website.

You have a budget of 50 actions you can perform on the website. Use it as efficiently as possible to test all important flows for the website and make the most accurate assessment of usability.

First, list the most important user goals for this site. Then, work through at least three distinct key flows that cover those goals, one by one. Use realistic placeholder data if you need to fill any fields.

If you are using a date picker widget in a booking flow, remember that the target date usually must be in the future. Today's date is {DATE}. If given the option, you should try to use a date picker instead of manually entering the date. The date picker control is usually located immediately to the right of the date field.

If you encounter errors, try your best to complete the flow but if you cannot complete it, move onto another flow.

Do not finish the trace until you have worked through the key flows you listed.

We made the following addition to the prompt for GPT-5-mini, as without it, it would try to access the internet:

IMPORTANT: Only interact with the website provided. Do not navigate to or visit any external websites or domains.

We appended the following prompt at the end of the interaction trace for a final usability score⁴:

Rate the website usability from 0 to 100.
The score should represent the probability that an average user can successfully use the website's most important flows.

You are testing a real website - the site you are testing is NOT a mock/demo/educational app. Review the assistant thoughts from each interaction step and create a list of usability problems. Each observation you make must reference a direct quote from the thought history, and you must rate it as a major or minor issue.

Pay special attention to broken navigation, failed page loads, non-responsive clicks, confusing flows, and unfinished core flows.

For example, look for thoughts that say "page appears to be stuck or not loading properly," "I need to wait briefly to ensure the page fully load", and "I need to wait for the page to load." If this appears multiple times, that is a red flag because the implementation might not contain that functionality and only appears to be loading.

If the thoughts contain "haven't successfully completed" that is also a red flag.

End with exactly one final line: Action: score(<number from 0 to 100>).

⁴This score was a prompt engineering strategy to ensure the model provided a bounded assessment.

C Study Instructions

We provided the participants with the following instructions before our study:

- You will be shown 30 pairs of websites.
- Your task is to evaluate which website between pairs of websites is more usable.
- These websites are clones of existing websites so many of the images are text placeholders. When voting, ensure you are evaluating the sites based on their usability.
- Occasionally websites may take awhile to load or may not reload. If that is the case feel free to move on.
- You must expand both websites using the expand ($\leftrightarrow$) button before you can vote.
- Browse each site freely – scroll, click links, and explore as you normally would.
- DO NOT enter in any personal information during your exploration. Instead, you can provide with fake information such as “John Doe, 123@demo.com, 123-456-7890”
- While you can choose ties, try and select one website over the other. There are no right or wrong answers – go with your honest impression.

We also show a sample screen from our study in Figure 9.

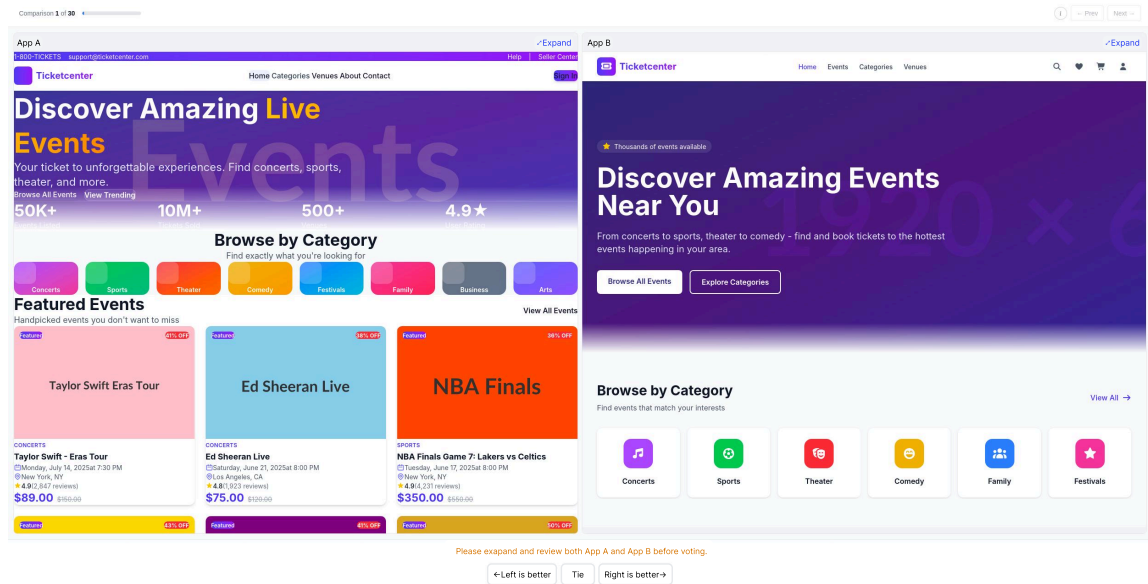


Fig. 9. A screen showing sample comparison participants made during our study. Participants were required to first expand both iframes to explore before they could vote.

D Model Hyperparameters

Model hyperparameters are provided in Table 2.

Table 2. Hyperparameters used to train open-source base models.

Base Model	Hyperparameter	Value
All	Effective batch size	8
	Epochs	1
	Learning Rate	2e-4
	LoRA	32
	Margin	
EvoCUA		15
GUI-Owl-1.5		12
Qwen3.5		15